과제 평가 질문 세트 만들기 (실습)

> 자신의 RAG 체인을 채점할 평가 질문 5개를 만들어 4개 이상 통과를 확인해봅니다.

chunk_size를 바꾸거나 하이브리드 가중치를 조정했을 때 좋아졌는지 나빠졌는지는 느낌으로 판단할 수 없습니다. 고정된 질문 세트로 매번 같은 시험을 치르게 해야 합니다.

오늘 만드는 `eval_set.json`은 하루짜리 산출물이 아닙니다. 3일차부터 6일차까지 매일 도전 미션의 마지막 5분에 그날의 검증 문항을 이 파일에 추가합니다. 7일차 평가 실습은 이 단일 파일을 입력으로 사용합니다.

정본은 하나만 둡니다. 5일차와 6일차 checkpoint에 동봉된 `eval_set.json`은 전날 미션을 못 따라온 경우에만 씁니다. 그냥 쓰면 그동안 누적한 문항이 날아갑니다.

파일 형식(스키마)은 `question`, `expected_source`, `type` 세 필드로 고정입니다.

- `question` : 실제 사용자가 물을 법한 질문입니다.
- `expected_source` : 정답의 근거가 되는 출처입니다. 오늘은 문서 파일명을 적습니다. 3일차부터는 같은 칸에 도구 이름, 기대 동작, Agent 이름이 들어옵니다. 여럿이면 쉼표로 이어 한 문자열로 적고, 문서에 답이 없는 질문만 null로 둡니다.
- `type` : 질문 유형입니다. 세 유형을 섞어야 좋은 시험지가 됩니다.
  - `fact` : 문서의 단어를 거의 그대로 쓴 질문입니다.
  - `paraphrase` : 문서에 없는 단어로 바꿔 물은 질문입니다. 의미 검색이 되는지 봅니다.
  - `no_answer` : 문서에 답이 없는 질문입니다. 거절하는지 봅니다.

따라하기

1. 아래 예시를 참고해 자신의 질문 5개로 `eval_set.json`을 day2_practice 폴더 바로 아래에 작성합니다. 세 유형을 반드시 섞습니다.

```
[
  {
    "question": "연차 이월은 며칠까지 가능한가요?",
    "expected_source": "docs/leave_policy.md",
    "type": "fact"
  },
  {
    "question": "월차 얼마나 받을 수 있어?",
    "expected_source": "docs/leave_policy.md",
    "type": "paraphrase"
  },
  {
    "question": "집에서 일하려면 어떻게 신청해?",
    "expected_source": "docs/remote_work_policy.md",
    "type": "paraphrase"
  },
  {
    "question": "해외 출장 식비 한도는 하루 얼마인가요?",
    "expected_source": "docs/travel_policy.md",
    "type": "fact"
  },
  {
    "question": "육아휴직은 얼마나 쓸 수 있나요?",
    "expected_source": null,
    "type": "no_answer"
  }
]
```

2. 자동 채점 스크립트를 `eval.py`로 저장하고 실행합니다.

```
# eval.py - 평가 세트 자동 채점
import json
from rag_chain import rag_chain, retriever

PASS_THRESHOLD = 4   # 2일차 성공 기준: 5문항 중 4문항 이상 정답

with open("eval_set.json", encoding="utf-8") as f:
    eval_set = json.load(f)

passed = 0
for item in eval_set:
    hits = retriever.invoke(item["question"])
    sources = sorted({h.metadata["source"] for h in hits})

    if item["expected_source"] is None:
        # no_answer 유형: 체인이 '찾을 수 없다'고 거절하는지로 채점
        answer = rag_chain.invoke(item["question"])
        ok = "찾을 수 없" in answer
    else:
        # 기대 출처가 검색 결과 상위 k개 안에 있는지로 채점
        ok = item["expected_source"] in sources

    passed += ok
    mark = "통과" if ok else "실패"
    print(f"[{mark}] ({item['type']}) {item['question']}")
    print(f"       기대: {item['expected_source']} / 실제 검색: {sources}")

print(f"\n정답 {passed}/{len(eval_set)}문항")
if passed >= PASS_THRESHOLD:
    print(f"[성공 기준 통과] {PASS_THRESHOLD}문항 이상 정답입니다.")
else:
    print(f"[성공 기준 미달] {PASS_THRESHOLD}문항 이상 필요합니다. {PASS_THRESHOLD -
passed}문항 더 맞혀야 합니다.")
```

채점 코드의 문법 두 가지만 짚고 갑니다. `with open(...) as f:` 블록은 블록을 벗어나는 순간 파일을 자동으로 닫습니다(Java의 try-with-resources와 같습니다).
`{h.metadata["source"] for h in hits}` 처럼 중괄호로 감싸면 중복이 없는 집합(set)이 됩니다. 같은 문서에서 청크가 여러 개 검색돼도 파일명을 한 번만 세기 위해 쓴 것입니다.

3. 4개 미만이면 아래 순서로 조정하며 다시 채점합니다.
    - k를 3에서 5로 올려 봅니다. 가장 싼 조치입니다.
    - 하이브리드 검색으로 바꿔 가중치를 조정합니다. paraphrase 실패에는 벡터 비중을, 고유명사 실패에는 BM25 비중을 올립니다.
    - 정답이 청크 경계에 잘렸다면 chunk_size와 overlap을 조정해 재적재합니다.
    - 하나를 바꿀 때마다 전체 세트를 다시 채점합니다. 한 질문을 고치려다 다른 질문이 깨지는 일이 흔합니다.

막힐 때

- no_answer 질문이 계속 실패합니다: 채점 문자열과 프롬프트의 거절 문구가 달라서인 경우가 많습니다. 4번째 세션 4-2 프롬프트의 '사내 문서에서 찾을 수 없습니다'와 `eval.py`의 "찾을 수 없" 문자열이 맞물리는지 확인합니다.
- 모든 질문에서 같은 파일만 검색됩니다: 청크 수를 먼저 확인합니다. 중복 적재로 특정 문서 청크가 비대해졌다면 chroma_db 폴더를 지우고 재적재합니다.

확인: 마지막 줄에 `[성공 기준 통과]`가 출력되면 성공입니다.

> 느낌이 아니라 고정된 평가 세트로 검색 품질을 측정하는 습관을 익혔습니다.

▼ 모범답안 (직접 해본 뒤 열어보세요)

같은 문서로 만든 다른 5문항입니다. 본문 예시와 겹치지 않게 골랐습니다.

```
[
  {"question": "미사용 연차는 며칠까지 이월되나요?", "expected_source": "docs/leave_poli
cy.md", "type": "fact"},
  {"question": "안 쓴 휴가 내년으로 넘길 수 있어?", "expected_source": "docs/leave_poli
cy.md", "type": "paraphrase"},
  {"question": "재택근무 신청서 양식 번호가 뭔가요?", "expected_source": "docs/remote_wo
rk_policy.md", "type": "fact"},
  {"question": "집에서 일하는 날 회사 망은 어떻게 붙어?", "expected_source": "docs/remot
e_work_policy.md", "type": "paraphrase"},
  {"question": "경조사 휴가는 며칠인가요?", "expected_source": null, "type": "no_answe
r"}
]
```

처음 돌리면 보통 4/5쯤 나옵니다. 조정 과정을 그대로 옮기면 이렇습니다.

```
k=3 기본값          -> 4/5 (paraphrase "집에서 일하는 날 회사 망" 실패)
k=5로 올림           -> 4/5 (변화 없음. 애초에 상위에 안 들어옴)
하이브리드 0.3/0.7 -> 5/5 (벡터 비중을 올리니 의역 질문이 잡힘)
```

문항을 고를 때 쓴 기준입니다.

- fact는 답이 한 문서에만 있는 걸 고릅니다. 여러 문서에 걸치면 채점이 애매해집니다.
- paraphrase는 문서에 없는 단어를 일부러 씁니다. 미사용 연차를 안 쓴 휴가로, VPN을 회사 망으로 바꾸는 식입니다.
- no_answer는 그럴듯한데 문서에 없는 주제로 잡습니다. 점심 메뉴처럼 동떨어진 질문은 너무 쉬워서 시험이 되지 않습니다.
